Rust

What does Rust have different from other programming languages.
- ownership and borrowing
- lifetime
- pattern matching
- concurrency without data races
- traits
- no null or garbage collection
- macros
- cargo
- community and ecosystem
- UTF-8 and Unicode text

Variables

Variables are declared using the `let` keyword.
Variable names are always in `snake_case`.

Changing Variables

Values fall into two types:
- mutable - the compiler will allow the variable to be written to and read from (`let mut var;`)
- immutable - the compiler will only allow the variable to be read from. (`let var;`)

Mutable values are denoted with a `mut` keyword

```
fn main() {
    let mut x = 42;
    x = 13;
}
```

Basic Data Types

Basic data types: **bool, u8**, **u16**, **u32**, **u64**, **u128**, **i8**, **i16**, **i32**, **i64**, **i128**, **usize**, **isize**, **f32**, **f64**

```
fn main() {
    let a: u32 = 1;    // u is only for +
    let b: i32 = -1;   // i is for ±
}
```

```
fn main() {
    let a = 13u8;
    let b = 7u32;
    let c = a as u32 + b;

    let t = true;
    println!("{}", t as u8);

    let d: u32 = 12;
    // let e: u32 = -12;
    // unsinged nubers include only positive numbers
    let e: i32 = 12;
    let f: i32 = -12;
    println!("{} {} {} {}",
             d, -(d as i32), e, f);
}
```

Basic Type Conversion

Luckily Rust makes numeric type conversions very easy with the `as` keyword.

```
fn main() {
    let a = 13u8;
    let b = 7u32;
    let c = a as u32 + b;
    println!("{}", c);

    let t = false;
    println!("{}", t as u8);
}
```

Constants

Constant names are always in `SCREAMING_SNAKE_CASE`.

```
const PI: f32 = 3.14159;

fn main() {
    println!(
        "To make an apple {} from scratch, you must first create a universe.",
        PI
    );
}
```

Collections

Rust provides a variety of collection types that allow you to store and manipulate data.

The main collections in Rust: **Arrays**, **Vectors**, **Slices**, **Strings**, **Tuple**, **Hash Maps**, **Hash Sets**

Functions associated with collections: `.iter()`, `.map()`

```
use std::collections::HashMap;
use std::collections::HashSet;

fn main() {
    let array: [i32; 3] = [1, 2, 3];
    println!("{:#?}", array);      // printing in debug mode

    let vector: Vec<i32> = vec![1, 2, 3];
    println!("{:?}", vector);

    let slice = &array[1..3];
    println!("{:?}", slice);

    let string: String = String::from("Hello, Rust!");
    println!("{:?}", string);

    let tuple: (i32, f64, u8) = (42, 3.14, 5);
    println!("{:?}", tuple);

    let mut map = HashMap::new();
    map.insert("one", 1);
    map.insert("two", 2);
    println!("{:?}", map);

    let mut set = HashSet::new();
    set.insert(1);
    set.insert(2);
    println!("{:?}", set);
}
```

Arrays

An *array* is a **fixed length** collection of data elements all of the same type.
The data type for an *array* is **`[T;N]`**
T = element's type
N = fixed length, known at compile time

```
fn main() {
   let nums: [i32; 5] = [1, 2, 3, 4, 5];
   println!("{}", nums[1]);

    println!("{:?}", nums);     // :? for displaying collections
    println!("{:#?}", nums); // :#? displaying a collection in debug mode

    for el in nums {    // could also be &nums
       print!("{} ", el);
   }

   for i in 0..=(nums.len() - 1) {
      print!("{} ", nums[i]);
   }

   for i in 0..nums.len() {
      print!("{} ", nums[i]);
    }

}
```

Vectors

Flexible collection.

```
fn main() {
   // vec![] is a macro
   let v1 = vec![1, 2, 3, 4, 5];
   println!("{:?}", v1);

   let mut v2 = Vec::new();
   v2.push(1);
   v2.push(2);
   v2.push(3);

   for el in &v1 {
      print!("{} ", el);
   }

   for el in v1.iter() {
      print!("{} ", el);
   }
   for i in 0..v1.len() {
      print!("{} ", v1[i]);
   }

   // we'll talk about Some and Result later
   if let Some(element) = v1.get(1) {
      println!("Second element: {}", element);
   } else {
      println!("Index out of bounds.");
   }

}
```

Strings

In Rust strings are UTF-8 encoded sequence of Unicode characters.
Rust also has a primitive string type, **`&str`**, which represents a string slice.

```
fn main() {
   let str1 = "Hello, world"; // &str Type
   println!("{:?}", str1);

   let mut s1: String = String::from("");
   s1.push_str("Hello");
   s1.push_str(", world!");
   println!("{:?}", s1);

   let s2: String = String::from("Rust");
   let s3: String = s1 + &s2;  // s1 will be moved to s3
   println!("{:?}", s3);
   // println!("{:?}", s1);   // doesn't work : s1 is borrowed by s3

   let s4: String = s3.clone();
   for ch in s4.chars() {
      print!("{}", ch);
   }

}
```

Tuple

`Tuple` represents an **immutable** collections of elements, that can have different data types.
Since it's **immutable**, it cannot be modified after its creation.
The elements of a tuple can be accessed by **index** (indexing starts from 0), using the **`.` operator**.

```
tuple_name.index
```

```
fn main() {
   // Create a tuple
   let my_tuple = (11, "Hello, Rust!", 3.14);

   // Access elements using indexing
   let first_element = my_tuple.0;
   let second_element = my_tuple.1;
   let third_element = my_tuple.2;
}
```

Functions

Hint: if you define a function, the data it accepts are called parameters. If you call that function and pass data to it, then it's called arguments

```
fn add(x: i32, y: i32) -> i32 {
   return x + y;
}

fn subtract(x: i32, y: i32) -> i32 {
   x - y
}

fn last_digit(mut x: u32) -> u8 {
   while x > 9 {
      x = x / 10;
   }
   return x as u8;
}
```

```
fn main() {
   let x: u32 = 4;
   println!("{} {} {}", x, last_digit(x), x);
}
```

Multiple Return Values - Tuple

```
fn swap(x: i32, y: i32) -> (i32, i32) {
  return (y, x);
}

fn main() {
  // return a tuple of return values
  let result: (i32, i32) = swap(123, 321);
  println!("{} {}", result.0, result.1);

  // destructure the tuple into two variables names
  let (a, b) = swap(result.0, result.1);
  println!("{} {}", a, b);
}
```

Returning Nothing

An empty tuple is represented by `()`

```
fn make_nothing() -> () {
  return ();
}
```

Infinite loop

```
fn main() {
  let mut x = 0;
  loop {
    x += 1;
    if x == 42 {
      break;
    }
  }
}
```

for

```
fn main() {
  // 0..5 does NOT include the LAST element
  // 0..=5 includes the LAST element
  for x in 0..5 {
    print!("{} ", x);
  }
}
```

match

The equivalent of the C switch statement match is exhaustive, so all cases must be handled.

```
fn main() {
  let x = 42;
  match x {
    0 => println!("zero"),
    // use a single `|` to separate multiple alternative patterns:
    1 | 2 => println!("1 or 2!"),
    3..=9 => println!("a number 3 to 9 inclusively"),

    matched_num @ 10..=100 => {
                println!("found {} number between 10 to 100!", matched_num);
    }

    101 => (), // do nothing if x equals 101
    _ => println!("The uncovered cases"),
  }
}
```

Matching Multiple Parameters

The match statements expects and compare tuples: elements inside ()

```
fn main() {
  let x = 42;
  let y = -101;

  // the program compiles
  // but the parenthesses are unnecessary for just an element
  match (x % 2 == 0) {
    (true) => println!("{} is an even number", x),
    false => println!("{} is an odd number", x),
  }
  match (x >= 0, y >= 0) {
    (true, true) => println!("Both numbers {} {} are positive", x, y),
    (true, false) => println!("Only {} is a positive number", x),
    (false, true) => println!("Only {} is a negative number", y),
    (false, false) => println!("Both numbers {} {} are negative", x ,y),
  }
}
```

Ternary expression (if)

```
fn main() {
  let x: i32 = 1;
  let s1: bool = x > 0;
  let s2: i32 = if x < 0 {-1} else {1};
}
```

Structs

```
struct SeaCreature {
  animal_type: String,
  name: String,
  arms: i32,
  legs: i32,
  weapon: String,
}

fn main() {
  // SeaCreature's data is on stack
  let ferris = SeaCreature {
    // String struct is also on stack,
    // but holds a reference to data on heap
    animal_type: String::from("crab"),
    name: String::from("Ferris"),
    arms: 2,
    legs: 4,
    weapon: String::from("claw"),
  };

  println!(
    "{} is a {}. They have {} arms, {} legs, and a {} weapon",
    ferris.name, ferris.animal_type, ferris.arms, ferris.legs, ferris.weapon
  );
}
```

Tuple-like Structs

For conciseness, you can create structs that are used like a tuple.

```
struct Location(i32, i32);
fn main() {
  let loc = Location(42, 32);  // This is still a struct on a stack
  println!("{}, {}", loc.0, loc.1);  // tuples are acces by . and index
}
```

trait

A trait contains at least one unimplemented method.
A trait can contain implemented methods.

```
trait MyTrait {
  fn foo(&self);
  ...
}
```

impl

Implement either a struct or a trait for a struct.
All unimplemented methods of the trait must be implemented.
&self / **&mut self** -> instance, accessing by **.**
Self -> return type, accessing for static methods by **::**

```
impl MyStruct {
  fn foo(&self) -> Self { ...
  }
}

impl MyTrait for MyStruct {    // implement all trait's methods
  fn foo(&self) -> Self { ...
  }
}
```

Dynamic vs Static Dispatch

Methods are executed in two ways:
- static dispatch - When the instance type is known, we have direct knowledge of what function to call.
- dynamic dispatch - When an instance type is not known, we must find out some way of calling the correct function.

Trait types **`&dyn MyTrait`** give us the ability to work with instances of objects indirectly using dynamic dispatch. When dynamic dispatch is used, Rust will encourage you to put **`dyn`** before your trait type so people are aware.

Memory details:
- Dynamic dispatch is slightly slower because of the pointer chasing to find the real function call.

enum

The value of an **enum** can be accessed using the operator **::**
To compare an **enum**, use **match** or **if let EnumName::Field = var**

```
enum Species {
  Crab,
  Octopus,
  Fish,
  Clam
}
// the enum can be written on a single line
enum Species { Crab, Octopus, Fish, Clam }
enum PoisonType { Acidic, Painful, Lethal }
enum Size { Big, Small }
enum Weapon {
  Price: u32,
  Id(u32),                // tuple
  // Id: u32              // rewritten
  Claw(i32, Size),        // tuple
  // Claw: (i32, Size)    // rewritten
  Poison(PoisonType),     // tuple
  // Poison: PoisonType,  // rewritten
  None
}
let v = Species::Crab;    // EnumName::Field
match v {
  Species::Crab => println!("crab"),
  _ => (),
}
if let Species::Crab = v {
  println!("crab");
}
```

Dynamic vs Static Dispatch

Trait types **`&dyn MyTrait`** give us the ability to work with instances of objects indirectly using dynamic dispatch.
When dynamic dispatch is used, Rust will encourage you to put **`dyn`** before your trait type so people are aware.

```
struct SeaCreature {
  pub name: String,
  noise: String,
}

impl SeaCreature {
  pub fn get_sound(&self) -> &str {
    &self.noise
  }
}

trait NoiseMaker {   // all traits methods must be implemented by impl
  fn make_noise(&self);
}

impl NoiseMaker for SeaCreature {
  fn make_noise(&self) {
    println!("{}", &self.get_sound());
  }
}

fn static_make_noise(creature: &SeaCreature) {
  // we know the real type
  creature.make_noise();
}

fn dynamic_make_noise(noise_maker: &dyn NoiseMaker) {
  // we don't know the real type
  noise_maker.make_noise();
}

fn main() {
  let creature = SeaCreature {
    name: String::from("Ferris"),
    noise: String::from("blub"),
  };
  static_make_noise(&creature);
  dynamic_make_noise(&creature);
}
```

Option

Rust has a built in generic enum called `Option` that allows us to represent nullable values without using `null`.

This enum is so common, instances of the enum can be created anywhere with the enum variants `Some` and `None`.

```
enum Option<T> {
  None,     // represintg nothing
  Some(T),
}
```

Result

Rust has a built in generic **enum** called **`Result`** that allows us to return a value that has the possibility of failing. It is the idiomatic way in which the language does error handling.

This enum is so common, instances of the enum can be created anywhere with the enum variants **`Ok`** and **`Err`**.

```
enum Result<T, E> {
   Ok(T), // the first one will be the success case
   Err(E), // the second one will be the error case
}
```

Pattern matching on error

Match lets us deconstruct Result elegantly and ensure we handle all cases!

```
 match result {
   Ok(v) => println!("found {}", v),
   Err(e) => println!("Error: {}",e),
}
```

Error Propagation `?`

`Result` is so common that Rust has a powerful operator `?` for working with them. These two statements are equivalent:

The operator **`?`** is used for error propagation in the context of `Result` types. It is a briefer way to handle error without writing

```
do_something_that_might_fail()?
// could be written
match do_something_that_might_fail() {
  Ok(v) => v,
  Err(e) => return Err(e),
}
```

Option/Result Handling: `unwrap`

Working with `Option`/`Result` can be tedious when you are just trying to write some quick code. Both `Option` and `Result` have a function called **`unwrap`** that can be useful for getting a value in a quick and dirty manner.

`unwrap` will:
1. **Get the value** inside Option/Result
2. If the enum is of type None/Err, **`panic!`**

These two pieces of code are equivalent for **Option**:

```
my_option.unwrap()
// is the equivalent of
match my_option {
  Some(v) => v,
  None => panic!("some error message generated by Rust!"),
}
```

Similarly, for **Result**:

```
my_result.unwrap()
// is the same as
match my_result {
  Ok(v) => v,
  Err(e) => panic!("some error message generated by Rust!"),
}
```

Use `match` when you can!

if let

A much easier way to handle error propagation in Rust is to use the `if let` block. It basically handles just the only **`match`** branch that we are interested to get.

For instance, if we want to get the **error**, **`if let Err(..) = ..`** will be used. Otherwise, **`if let Ok(..) = ..`** will be chosento handle the **success** case.

`if let` can be used only for **`Result` and `Option`** data tyes.

```
fn div(x: f64, y: f64) -> Result<f64, String> {
  if y == 0.0 {
     return Err(String::from("Invalid operation: division by zero."));
  }
  return Ok(x / y);
}

fn main() {
  if let Ok(res) = div(10.0, 1.0) {
     println!("{}", res);      // will be displayed
  }
  if let Err(err) = div(10.0, 1.0) {
     println!("{}", err);      // will NOT be displayed
  }

  if let Ok(res) = div(10.0, 0.0) {
     println!("{}", res);      // will be displayed
  }
  if let Err(err) = div(10.0, 0.0) {
     println!("{}", err);      // will NOT be displayed
  }
}
```

Extracitng a Result

Another brilliant way to extract a **`Result<T, E> { Ok(T), Err(E), }`** is to use the functions **`.ok()`** and **`.err()`**, representing the specific variant.

These functions will return an `Option`:
- `Some` if the Result function had something to return, for results like `Result<i32, i32>`.
- `None` if the Result function returned either a `Err(())` or an `Ok(())`, for results like `Result<(), ()>`, `Result<i32, ()>` or `Result<(), i32>`

```
// Using .ok() will extract the Ok variant
if let Some(value) = good1.ok() {
 println!("Result (Ok): {}", value);
} else {
 println!("Result (Err)");
}

// Using .err() to extract the Err variant
if let Some(err) = bad1.err() {
 println!("Error: {}", err);
} else {
 println!("No error (Ok)");
}

 match result {
 Ok(v) => println!("found {}", v),
 Err(e) => println!("Error: {}",e),
}
```

Borrowing Ownership `&`

References allow us borrow access to a resource with the `&` operator.
References are also dropped like other resources.
let a: i32 = 23; // it will store the value 23
let b: **&**i32 = **&**a; // it will store the address in RAM memory of `a`

Dereferncing

Using `&mut` references, you can set the owner's value using the `*` operator.

You can also get a copy of an owned value using the `*` **operator** (if the value can be copied - we will discuss copyable types in later chapters).

```
let mut foo = 42;
let f = &mut foo;
let bar = *f; // get a copy of the owner's value
*f = 13;      // set the reference's owner's value
```

Functions That Change Vars `&mut`

```
// without &mut, the variable values cannot be changed
fn swap(a: &mut i32, b: &mut i32) {
   let temp = *a;
   *a = *b;
   *b = temp;
}

fn main() {
   let mut x: i32 = 5;
   let mut y: i32 = 10;
   swap(&mut x, &mut y);
   println!("{} {}", x, y);    // 10 5
}
```

Passing Borrowed Data

```
// without &mut, the variable's value cannot be modified
fn do_something(x: &mut u32) {
   *x += 1;
    // mutable reference f is dropped here
}

fn main() {
   let mut x: u32 = 12;
   do_something(&mut x);
   // because all mutable references are dropped within
   // the function do_something, we can create another.
   do_something(&mut x);
   println!("{}", x); // prints 14
   // foo is dropped here
}
```

Accessing After Borowing

No many how many references are passed from the borrowed variable, its value can be accessed without having to dereferencing it.

```
let x: u32 = 14;
let mut y: &&&u32 = &&&x;
println!("{} = {}", x, y);
```

Refernces of References

References can even be used on pieces of references.

```
struct Foo {
   x: i32,
}

fn do_something(a: &Foo) -> &i32 {
   return &a.x;
}

fn main() {
   let mut foo = Foo { x: 42 };
   let x = &mut foo.x;
   *x = 13;
   // x is dropped here allow us to create a non-mutable reference
   let y = do_something(&foo);
   println!("{}", y);
   // y is dropped here
   // foo is dropped here
}
```

Explicit Lifetimes `'a`

Even though Rust doesn't always show it in code, the compiler understands the lifetime of every variable and will attempt to validate that a reference never exists longer than its owner.

Functions can be explicit by parameterizing the function signature with symbols that help identify which parameters and return values share the same lifetime.

Lifetime specifiers always start with a ' (e.g. `'a`, `'b`, `'c`)

```
struct Foo {
   x: i32,
}

// the parameter foo and return value share the same lifetime
fn do_something<'a>(foo: &'a Foo) -> &'a i32 {
   return &foo.x;
}

fn main() {
   let mut foo = Foo { x: 42 };
   let x = &mut foo.x;
   *x = 13;
   // x is dropped here, allowing us to create a non-mutable reference
   let y = do_something(&foo);
   println!("{}", y);
   // y is dropped here
   // foo is dropped here
}
```

#Multiple Lifetimes

Lifetime specifiers allow us to be explicit with certain scenarios the compiler cannot resolve itself by distinguishing all of a function signature component's lifetimes.

```
struct Foo {
   x: i32,
}

// foo_b and the return value share the same lifetime
// foo_a has an unrelated lifetime
fn do_something<'a, 'b>(foo_a: &'a Foo, foo_b: &'b Foo) -> &'b i32 {
   println!("{}", foo_a.x);
   println!("{}", foo_b.x);
   return &foo_b.x;
}

fn main() {
   let foo_a = Foo { x: 42 };
   let foo_b = Foo { x: 12 };
   let x = do_something(&foo_a, &foo_b);
   // foo_a is dropped here because only foo_b's lifetime exist beyond here
   println!("{}", x);
   // x is dropped here
   // foo_b is dropped here
}
```

Lifetimes In Data Types

Similarly to functions, data types can be parameterized with lifetime specifiers of its members.
Rust validates that the containing data structure of the references never lasts longer than the owners its references point to.
We can't have structs running around with references pointing to nothingness!

```
struct Foo<'a> {
   i:&'a i32
}

fn main() {
   let x = 42;
   let foo = Foo {
      i: &x
   };
   println!("{}",foo.i);
}
```

Static Lifetimes

A **static variable** is a memory resource created at compile-time that exists through a program start to finish. They must have their types explicitly specified.

A **static lifetime** is a memory resource that lasts indefinitely to the end of a program. Note that by this definition some static lifetime resources can be created at runtime.

Resources with static lifetimes have a special lifetime specifier `'static`.

`'static` resources will never drop.

If static lifetime resources contain references they must all be `'static` (anything less would not live long enough).

Memory detail:

- Modifying static variables is inherently dangerous because they are globally accessible to be read from by anyone introducing the possibility of a data race. We'll talk about the challenges of global data later.
- Rust allows the use of `unsafe { ... }` blocks to perform some operations that the compiler cannot make memory guarantees about.

```
static PI: f64 = 3.1415;

fn main() {
   // static variables can also be scoped to a function
   static mut SECRET: &'static str = "swordfish";

   // string literals have a 'static lifetime
   let msg: &'static str = "Hello World!";
   let p: &'static f64 = &PI;
   println!("{} {}", msg, p);

   // You can break some rules, but you must be explicit
   unsafe {
      // we can set SECRET to a string literal because it is also `static
      SECRET = "abracadabra";
      println!("{}", SECRET);
   }
}
```

Generic Functions

Generics in Rust work hand in hand with **traits**. When we describe a **parameterized type** `T` we can constrain what types can be used as an argument by listing what required traits the argument must implement.

In this example type `T` must implement trait `Foo`:

```
fn my_function<T>(foo: T)
where     // expects a TRAIT, not a datatype or a struct
  T: Foo
{
  ...
}
```

By using generics we create static typed functions at compile time that will have known types and sizes, allowing us to perform static dispatch and store as a sized value.

Generic Functions Shorthand

```
fn my_function(foo: impl Foo) {
  ...
}

// This is equivalent to writing

fn my_function<T>(foo: T)
where    // expects a TRAIT, not a datatype or a struct
  T:Foo
{
  ...
}
```

Simple Generic Function

```
fn add<T>(a: T, b: T) -> T
where
  T: std::ops::Add<Output = T>,
{
  // equivalnt of : return (a + b);
  a + b
}

fn main() {
  let a: f64 = add(1.0 as f64, 11.0 as f64);

  let b: i32 = add(-1.0 as i32, -11.0 as i32);
  let c: isize = add(-1.0 as isize, -11.0 as isize);

  let d: u32 = add(1.0 as u32, 11.0 as u32);
  let e: usize = add(1.0 as usize, 11.0 as usize);

  println!("{} {} {} {} {}",
        a, b, c, d, e);
}
```

Box

`Box` is a data structure that allows us to move our data from the stack to the heap.

`Box` is a struct known as a smart pointer that holds the pointer to our data on the heap.

Because **`Box`** is a struct with a known size (because it just holds a pointer), it is often used as a way to store a reference to something in a struct that must know the size of its fields.

`Box` is so common it can be used from anywhere:

```
Box::new(Foo { ... })
```

Heap Allocated Memory `Box`

`Box` is a smart pointer that lets us move data from the stack to the heap.

Dereferencing it lets us use the heap allocated data ergonomically as if it were the original type.

```
struct Pie;

impl Pie {
  fn eat(&self) {
    println!("tastes better on the heap!")
  }
}

fn main() {
  let heap_pie = Box::new(Pie);
  heap_pie.eat();
}
```

Referencing Counting `Rc`

`Rc` is a smart pointer that moves data from the stack onto the heap. It allows us to clone other **`Rc`** smart pointers that all have the ability to immutably borrow the data that was put on the heap.

Only when the last smart pointer is dropped does the data on the heap become deallocated.

Closure (Anonymous Functions)

Do you miss the **`lambda expression`** from other languages?

Rust has **`closures`** are similar to functions, but have a more compact syntax. A **`closure`** is a feature that allows you to define an **anonymous function** in a flexible and concise way.
- **`| parameter1, parameter2, ...| expression`** is used to define a small, inline function.
- **`| x |`** part declares a paramaters, in this case, **x**.

```
fn main() {
  let add = |x, y| x + y;          // Rust closure (anonymous function)
  let result = add(3, 5);
  println!("Result: {}", result);
}
```

Referencing Counting

```
use std::rc::Rc;

struct Pie;

impl Pie {
   fn eat(&self) {
      println!("tastes better on the heap!")
   }
}

fn main() {
   let heap_pie = Rc::new(Pie);
   let heap_pie2 = heap_pie.clone();
   let heap_pie3 = heap_pie2.clone();

   heap_pie3.eat();
   heap_pie2.eat();
   heap_pie.eat();

   // all reference count smart pointers are dropped now
   // the heap data Pie finally deallocates
}
```

Sharing Access `RefCell`

`RefCell` is a container data structure commonly held by smart pointers that takes in data and lets us borrow mutable and immutable references to what's inside. It prevents borrowing from being abused by enforcing Rust's memory safety rules at runtime when you ask to borrow the data within:

Only one mutable reference OR multiple immutable references, but not both!

If you violate these rules `RefCell` will panic.

Sharig Across Threads

`Mutex` is a container data structure commonly held by smart pointers that takes in data and lets us borrow mutable and immutable references to the data within. This prevents borrowing from being abused by having the operating system restrict only one CPU thread at time to have access to the data, blocking other threads until that original thread is done with its locked borrow.

Multithreading is beyond the scope of Tour of Rust, but **`Mutex`** is a fundamental part of orchestrating multiple CPU threads accessing the same data.

There is a special smart pointer `Arc` which is identical to `Rc` except uses thread-safe incrementing of reference counts. It's often used to have many references to the same `Mutex`.

Sharing Access `RefCell`

```
use std::cell::RefCell;

struct Pie {
   slices: u8
}

impl Pie {
  // without &mut self, the instance cannot be changed
   fn eat(&mut self) {
      println!("tastes better on the heap!");
      self.slices -= 1;
   }
}

fn main() {
   // RefCell validates memory safety at runtime
   // notice: pie_cell is not mut!
   let pie_cell = RefCell::new(Pie{slices:8});

   {
      // but we can borrow mutable references!
      let mut mut_ref_pie = pie_cell.borrow_mut();
      mut_ref_pie.eat();
      mut_ref_pie.eat();

      // mut_ref_pie is dropped at end of scope
   }

   // now we can borrow immutably once our mutable reference drops
   let ref_pie = pie_cell.borrow();
   println!("{} slices left",ref_pie.slices);
}
```

Sharig Across Threads

```
use std::sync::Mutex;

struct Pie;

impl Pie {
   fn eat(&self) {
      println!("only I eat the pie right now!");
   }
}

fn main() {
   let mutex_pie = Mutex::new(Pie);
   // let's borrow a locked immutable reference of pie
   // we have to unwrap the result of a lock
   // because it might fail
   let ref_pie = mutex_pie.lock().unwrap();
   ref_pie.eat();
   // locked reference drops here, and mutex protected value can be used by someone else
}
```

Collection Iterator

In Rust, an iterator is an abstraction that decomposed a collection into a sequence of values.
You've seen an iterator before: in the **`for`** loop

```
// 0..5 iterates the values from 0 to 4
for i in 0..5 {
   println!("{}", x);
}
```

`.iter()`: This method is used to create an iterator over a collection that **borrows** each element

`.into_iter()`: This method is used to create an iterator that **takes ownership** of each element. This can be useful when you want to **move or transfer ownership** of the elements to another part of your code.

```
fn main() {
   let numbers = vec![1, 2, 3, 4, 5];
   // each element will be modified
   let doubled: Vec<i32> = numbers
      .iter()        // generates a sequence of elements
                     // borrows each element from the initial collection
      .filter(|x| *x % 2 == 0) // filter even numbers
                               // expects a closure, |x| = paramter
      .map(|x| x * 2)    // apply a function to each element
                         // exepcts a closure, |x| = paramter
      .collect();   // the modification will be returned
                    // only if you COLLECT !!!!!

   // generating a sequence with all elements
   for &el in doubled.iter() {
      print!("{} ", el);
   }
   println!();
}
```

For Range

In Rust, the `for` loop is often used with `ranges` to **iterate** over a sequence of values.

```
fn main() {
   // NOTHING: the start value > stop value
   for i in 1..-2 { // -2..1 => -2 -1 0 1
      println!("{}", i);
   }

   // Create a range from -5 to 5 (inclusive)
   let my_range: Vec<_> = (-5..=5)
      .rev()              // Reverse the range
      .step_by(2)         // Jumps over each 2 elements
                          // its argument must be POSITIVE
      .filter(|&x| x % 2 == 0)       // Filter for even numbers
                          // filter expects a closure, |x| = parameter
      .map(|x| x * x)     // Map (modify) each element to its square
                          // map expects a closure, |x| = parameter
                          // Apply a function to each element
      .enumerate()        // Enumerate to get both index and value
      .collect();         // Collect the results into a vector
                          // The modified range will be returned
                         // only if you use COLLECT !!!!!!!!!!
   // Print the final result
   for (index, value) in my_range {
      println!("Index {}: {}", index, value);
   }
}
```